REPORT — agent-hands

As-built summary

Ashish Kosana

19 August 2026

Contents

Architecture

Single process, four independently testable components with one deliberate seam. The **Surface** (`surface.py`) owns everything browser-specific: perceiving state, resolving locators, acting. Above it, everything speaks in artifact terms. The **planner** (`planner.py`) is an LLM tool-use loop that can only act by pointing — every observation enumerates actionable elements with refs, and the act tool takes a ref, so the model cannot free-form a click. The **recorder** (`recorder.py`) distills the run *as it happens*: every candidate locator rung is round-tripped through the same engine replay will use, against the live page at act time, and kept only if it resolves uniquely to the exact element acted on. The **replay engine** (`replay.py`) consumes only the artifact — a hermetic test (fresh subprocess, keys stripped, non-loopback sockets blocked) proves no model is anywhere in that path.

Key decisions: *contract-first discovery* — the caller-facing contract (parameters, outputs, outcome codes) is declared in a request file before any model runs; the LLM fills in the *how*, never the *what*. *The model proposes, the recorder verifies* — checkpoint headings, identity anchors, output anchors, and outcome markers all come from the model but are mechanically proven against the live page (and rejected if they embed run-specific values) before entering the artifact. *Supervised discovery* — the request declares the expected output value for the example invocation; an anchor whose extraction doesn't match is refused, so a wrong anchor cannot become a capability that returns wrong balances forever. Trade-off accepted: discovery is slower and stricter than a naive recorder; every check exists to keep a defect out of the thing that runs a million times.

The provider seam is one thin client for any OpenAI-compatible endpoint; the committed evidence run used a free-tier-hosted open-weights model deliberately — discovery succeeding on a modest model is evidence the artifact, not the model, carries the reliability.

Artifact schema

JSON on disk, Pydantic-modeled, diffable (`docs/DESIGN.md` shows a full example that is load-tested against the real schema). The caller's contract is top-level and closed: typed `parameters`, typed `outputs` (with parse specs — a strict en_US money parser *fails* on a comma-decimal rendering rather than mis-parsing it 100×), and an enumerable `outcomes` map validated for closure both ways (every recognizer maps to a declared code; every declared code is reachable). Steps carry intent, a **locator ladder** (role+name → label

association → exact text → geometric relation → CSS as a flagged, fragile last resort; 0 matches → next rung, >1 → refuse), a recorded **fingerprint** verified on every replay (uniqueness is not correctness), pre/postconditions, and a risk label. **Recognizers** are scoped (armed per step range), structural (named regions, never bare page text), and provenance-tracked — the `MEMBER_NOT_FOUND` recognizer exists because a second discovery run *actually saw* a member not be found. The **checkpoint must bind identity**: load-time validation refuses a parameterized capability whose checkpoint proves only "a details page loaded" rather than "*this member's* page loaded" — the confidently-wrong-answer bug, prevented structurally.

Determinism & error handling

Replay uses no model, no coordinates, and no sleeps: bounded budgets with postcondition polling; Playwright gets short per-attempt timeouts inside an engine-owned step budget, and recognizers are checked before and between attempts, so a known interstitial appearing *between* steps is recovered, not smashed into an opaque timeout. The result contract is explicitly separated: **business outcomes** (not-found, permission denied, server-side validation) are answers with auditable match evidence; **recoverable conditions** carry real recovery steps, an explicit resume point, and per-run fire caps — exhaustion or a broken recovery *promotes to FAILURE naming the original condition*; **hard failures** report step, expected vs observed, screenshot and accessibility snapshot. Two asymmetries matter: freshness suppression (a match present *before* an action may not classify it) applies only to business outcomes — a blocking dialog that predates your action is exactly what recovery is for; and risky steps are never auto-retried after their action ran (the double-fired-transfer scenario) — they escalate. Drift, secondarily: a renamed label exhausts the ladder and fails loudly listing what was tried (measured in the eval table); per-step rung-hit telemetry in every trace is the early-warning signal.

Heterogeneity & multi-tenant

The seam is the Surface protocol: everything above it addresses targets by role + accessible name, geometric relations ("cell right of 'Savings'"), and surface-neutral context paths — exactly the vocabulary desktop accessibility APIs (UIA/AX) expose. What transfers: the ladder discipline, recognizers, checkpoints, typed I/O, the control-token escalation model. What needs per-surface work: a role/rung mapping layer (ARIA `textbox` ? UIA `Edit`; frame chain ? window/pane chain; the CSS rung declared web-only), input synthesis, and — honestly — name computation: browsers *compute* accessible names from

adjacent text; a custom-drawn Win32 teller app exposes an empty tree, and the realistic fallback there is an OCR/vision rung, rejected as a primary mechanism for web but acceptable as a flagged last rung on desktop.

Multi-tenant reuse (designed, not built): tenant variance is a typed **overlay patch** keyed by artifact addresses (step+rung, condition, checkpoint, output anchor) with explicit merge semantics, pinned to a base `{capability, version}` and hard-failing on mismatch; overlays may only map to already-declared outcome codes — a new business outcome is a contract change and bumps the base version for every caller. Structural variance (an extra verification step) is a capability *variant* behind a tenant→(capability, version) resolution table. Drift detection is the existing rung telemetry aggregated per tenant×capability into a health file with a threshold that flags re-discovery before replay breaks. The allowlist deliberately lives in policy config, never in the overlay — a tenancy mechanism must not widen a safety boundary.

Escalation & handoff

"Stuck" is detected as: an unrecognized blocking state, a recovery that cannot proceed safely (including the risky-retry prohibition), or an operator pressing *Request control* mid-run. The control token is **engine-owned**: the operator console (an HTTP thread in the same process, which never touches the thread-affine browser API) only posts transition requests; the engine polls at every tick, parks, and acknowledges — the console shows control as granted only after that acknowledgment, so two drivers on one session is structurally impossible. The intervention request carries capability, step, reason, non-sensitive params, rendered expectations, the last N trace events, and a screenshot. The state machine gives the human four real exits: take control (drive the **same live session** — headed directly, or headless via a minimal command channel the parked engine executes on their behalf), hand back, **abort**, or **resolve as a declared business outcome** (undeclared codes are refused) — every transition attributed to an operator identity. Handback resumes via a **forward position scan** (checkpoint first, then the furthest step whose postconditions hold) because humans finish the screen they're on — blind current-step retry would re-execute against a moved-on page; if the only resume option is re-running a risky step, it refuses. Unanswered interventions expire (TTL) and close the session. Human actions are captured by listeners injected at context creation as semantic descriptors with **masked lengths, never values**; a canary test types a sentinel during handoff and asserts it

appears nowhere on disk — it caught a real leak (the failure snapshot) during development, which is why evidence capture is suppressed once a human has driven the session. A live captured handoff is in `evidence/escalation-run/`.

Safety

Allowlist enforcement is at the **network layer** (context-wide request routing: clicks, redirects, popups, subresources), defaulting to exactly the artifact's entry host; blocked traffic surfaces as `POLICY_VIOLATION`, not a mysterious timeout. Risk is **mutating-by-default**: any action observed to fire a non-GET request during discovery (watched through the page's reaction, not just the driver call) is recorded risky; signals only ever raise risk. The gate has teeth: risky steps do not replay unattended until a human signs the artifact (`hands review`), and the signature binds to the artifact hash — any edit or re-record voids it. Secrets are symbolic end-to-end: models see parameter *names*; sensitive values come from the environment and resolve inside the surface at act time; password-field values are masked in observations; sensitive params and outputs are masked in traces and transcripts; failure reports mask sensitive extractions. Limits, honestly: page-native PII (names, balances rendered by the app) would reach the model during discovery on a real system — the mitigation is containment (discovery is a privileged, once-per-capability event; replay sends nothing to any model) plus zero-retention provider terms, not magic redaction; and prompt injection from page text is mitigated by grounding (act-by-ref, mechanical verification of every model-supplied string, data-independence checks) and fencing, not eliminated.

Cuts

Cut, in the order published in the design: (1) the public-site portability leg; (2) the second tenant skin — which breaks my own "never cut both" rule, acknowledged: with a fixed budget I chose depth on escalation, policy, and measured evals (top-weighted criteria) over a second self-authored variant; generalization evidence is instead the drift eval, rung telemetry, and the surface-neutral schema. Also deliberately thin: the operator console UI (the control-transfer model is real; the skin is minimal), discovery-time escalation (replay-side is fully wired; a stuck planner ends the run rather than raising an intervention — same hub, one integration short), and element-level taint masking (evidence under human control is suppressed wholesale instead). Next, in order: overlay loading + a structurally divergent second tenant to

make §4 demonstrated rather than argued; discovery-side escalation; taint-based evidence masking; a capability catalog endpoint (the schema already doubles as a tool definition).